

Assignment 7

Exercice 1. *Hessian of functions.*

1. Determine the Hessians of the functions defined on page 4 and implement them.
2. Write a function whose input is a symmetric matrix, and such that :
 - the function computes the eigenvalues of the matrix. You can use the function `numpy.linalg.eigvalsh()` to compute the eigenvalues.
 - the function returns the highest and lowest eigenvalues of the matrix.
3. Write a function which receives the highest and lowest eigenvalues λ_n and λ_1 of a symmetric matrix. The function prints λ_1, λ_n , and a message according to the following conditions :
 - If $\lambda_1 > 0$, then print **Definite positive matrix**.
 - If $\lambda_n < 0$, then print **Definite negative matrix**.
 - If $\lambda_1 < 0$ and $\lambda_n > 0$, then print **Indefinite matrix**.

Exercice 2. *Joined gradient descent and Newton method.*

The function to implement receives the following parameters :

- The objective function $f: \mathbb{R}^n \rightarrow \mathbb{R}$
- The gradient ∇f of the objective function
- The Hessian $\nabla^2 f$ of the objective function
- The initial point $\mathbf{x}_0$
- A maximum number of iterations N
- A threshold $\tau > 0$
- The parameters α_{init}, c_1, ρ , and a maximum number of iterations N_b for the backtracking algorithm.

The function to implement performs the following steps :

Fix $\mathbf{m} = \mathbf{0}$.

For $\mathbf{k} = 0, \dots, N - 1$, do the following steps :

1. Compute the gradient $\mathbf{g}_k$ at the point $\mathbf{x}_k$, i.e $\mathbf{g}_k = \nabla f(\mathbf{x}_k)$.
2. If $\|\mathbf{g}_k\| < \tau$, do **res** = 1 and stop the loop.
3. If the stopping criterion is not fulfilled, compute the Hessian at $\mathbf{x}_k$: $\mathbf{B}_k = \nabla^2 f(\mathbf{x}_k)$.
4. Try to perform a Cholesky decomposition of $\mathbf{B}_k$:
 - If the decomposition holds, we obtain $\mathbf{B}_k = LL^T$. Then, solve the system $LL^T \mathbf{p}_k = -\mathbf{g}_k$ to obtain the

descent direction $\mathbf{p}_k$.

- If the factorization fails, define the descent direction as $\mathbf{p}_k = -\mathbf{g}_k$, then do $\mathbf{m} = \mathbf{m} + 1$.

5. Compute the step size α_k using the backtracking Algorithm with the condition of sufficient descent.

6. Compute the updating of the sequence $\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{p}_k$.

Return $\mathbf{x}_k$, $\mathbf{g}_k$, $\mathbf{m}$ and $\mathbf{res}$.

Remark : To compute the Cholesky factorization and solve the system of equations, you can use the functions `scipy.linalg.cho_factor` and `scipy.linalg.cho_solve`. If the matrix is not positive definite, the function `scipy.linalg.cho_factor` returns the exception `scipy.linalg.LinAlgError`. You can manage this error by defining the direction descent as $-\mathbf{g}_k$.

Your task :

1. Implement the Algorithm described above. If the objective function has 2 variables, make the Algorithm return the list all the points $\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_k$ to be able to use them later on to plot the trajectory.

2. Test the Algorithm on the functions and initial points $\mathbf{x}_0$ defined on page 4, where :

- The maximum number of iterations is $N = 10000$.

- The threshold $\tau = \sqrt{n\epsilon_m}$, where ϵ_m is the machine epsilon and n is the number of variables of f .

- For the backtracking algorithm, use $\alpha_{init} = 1$, $c_1 = 0.1$, $\rho = 0.6$, and the maximum number of iterations is $N_b = 100$.

Print $f(\mathbf{x}_0)$ and the following values at each iteration :

- the iteration number k .

- the number m which determines the number of times $\mathbf{B}_k$ is not positive definite.

- the value $\mathbf{res}$.

- the norm of the gradient $\|\mathbf{g}_k\|$.

- If $n \leq 6$, print the vector $\mathbf{x}_k$. Otherwise, print the first and last three components of $\mathbf{x}_k$.

- $f(\mathbf{x}_k)$

If $n = 2$, plot the level lines of the function and the trajectory defined by all the points $\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_k$ generated by the Algorithm.

3. Based on the value of the parameter $\mathbf{m}$, determine whether the Algorithm behaves more as a gradient descent or as a Newton method.

Exercice 3. Strongly convex functions and quasi-Newton methods.

1. Show that if $g \in C^1(\mathbb{R}^n)$ is convex, then

$$(\nabla g(x) - \nabla g(y))^T(x - y) \geq 0 \quad \forall x, y \in \mathbb{R}^n.$$

2. Deduce from 1. that a strongly convex function $f \in C^1(\mathbb{R}^n)$ of parameter $m > 0$ satisfies that

$$(\nabla f(x) - \nabla f(y))^T(x - y) \geq m\|x - y\|^2 \quad \forall x, y \in \mathbb{R}^n.$$

3. Deduce from 2. a property of quasi-Newton methods applied to strongly convex functions.

Exercise 4. Modified BFGS Algorithm.

Algoritmo 1: Método BFGS modificado

Entrada: La función $f(\mathbf{x})$, su gradiente $\nabla f(\mathbf{x})$, un punto inicial $\mathbf{x}_0$, una tolerancia $\tau > 0$, una aproximación $\mathbf{H}_0$ de la inversa de la Hessiana de f en el punto $\mathbf{x}_0$, el número máximo de iteraciones N y los parámetros para el algoritmo de backtracking.

```

1 Calcular  $\mathbf{g}_0 = \nabla f(\mathbf{x}_0)$ ;
2 Definir  $\mathbf{I}$  como la matriz identidad y  $res = 0$ ;
3 for  $k = 0, 1, 2, \dots, N-1$  do
4   if  $\|\mathbf{g}_k\| < \tau$  then
5     Hacer  $res = 1$  y terminar el ciclo;
6   end
7   Calcular la dirección  $\mathbf{p}_k = -\mathbf{H}_k \mathbf{g}_k$ ;
8   if  $\mathbf{p}_k^T \mathbf{g}_k > 0$  then
9     Calcular  $\lambda_1 = 10^{-5} + \frac{\mathbf{p}_k^T \mathbf{g}_k}{\mathbf{g}_k^T \mathbf{g}_k}$ ;
10    Reemplazar  $\mathbf{H}_k$  por  $\mathbf{H}_k + \lambda_1 \mathbf{I}$ ;
11    Redefinir  $\mathbf{p}_k$  como  $\mathbf{p}_k - \lambda_1 \mathbf{g}_k$ ;
12  end
13  Calcular  $\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{p}_k$ , donde  $\alpha_k$  se obtiene con el algoritmo de backtracking;
14  Calcular  $\mathbf{s}_k = \mathbf{x}_{k+1} - \mathbf{x}_k$ ;
15  Calcular  $\mathbf{y}_k = \mathbf{g}_{k+1} - \mathbf{g}_k$ ;
16  if  $\mathbf{y}_k^T \mathbf{s}_k \leq 0$  then
17    Calcular  $\lambda_2 = 10^{-5} - \frac{\mathbf{y}_k^T \mathbf{s}_k}{\mathbf{y}_k^T \mathbf{y}_k}$ ;
18    Definir  $\mathbf{H}_{k+1} = \mathbf{H}_k + \lambda_2 \mathbf{I}$ ;
19  else
20    Calcular  $\rho_k = \frac{1}{\mathbf{y}_k^T \mathbf{s}_k}$ ;
21    Definir  $\mathbf{H}_{k+1} = (\mathbf{I} - \rho_k \mathbf{s}_k \mathbf{y}_k^T) \mathbf{H}_k (\mathbf{I} - \rho_k \mathbf{y}_k \mathbf{s}_k^T) + \rho_k \mathbf{s}_k \mathbf{s}_k^T$ ;
22  end
23 end
24 Devolver  $\mathbf{x}_k, \mathbf{g}_k, k, res$ ;
```

FIGURE 0.1: Modified BFGS Algorithm

1. Implement the modified BFGS Algorithm. If the objective function has 2 variables, make the Algorithm return the list all the points $\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_k$ to be able to use them later on to plot the trajectory.

2. Test the Algorithm on the functions and initial points $\mathbf{x}_0$ defined on page 4, where :

- $\mathbf{H}_0$ is the Identity.
- The maximum number of iterations is $N = 10000$.
- The threshold $\tau = \sqrt{n} \epsilon_m$, where ϵ_m is the machine epsilon.
- For the backtracking algorithm, use $\alpha_{init} = 1$, $c_1 = 0.1$, $\rho = 0.6$, and the maximum number of iterations is $N_b = 100$.

Print $f(\mathbf{x}_0)$ and the following values at each iteration :

- the iteration number $\mathbf{k}$.
- the norm of the gradient $\|\mathbf{g}_k\|$.
- If $n \leq 6$, print the vector $\mathbf{x}_k$. Otherwise, print the first and last three components of $\mathbf{x}_k$.
- $f(\mathbf{x}_k)$

If $n = 2$, plot the level lines of the function and the trajectory defined by all the points $\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_k$ generated by the Algorithm.

Test functions and initial points

1. Beale function

$$f(x_1, x_2) = (1.5 - x_1 + x_1 x_2)^2 + (2.25 - x_1 + x_1 x_2^2)^2 + (2.625 - x_1 + x_1 x_2^3)^2$$

with $\mathbf{x}_0 = (2, 3)$.

2. Himmelblau function

$$f(x_1, x_2) = (x_1^2 + x_2 - 11)^2 + (x_1 + x_2^2 - 7)^2$$

with $\mathbf{x}_0 = (2, 4)$.

3. Hartmann function of dimension 6

$$f(x_1, \dots, x_6) = -\frac{1}{1.94} \left[2.58 + \sum_{i=1}^4 \alpha_i \exp \left(-\sum_{j=1}^6 a_{ij} (x_j - p_{ij})^2 \right) \right]$$

where

$$\alpha = (1, 1.2, 3, 3.2)$$
$$A = (a_{ij}) = \begin{pmatrix} 10 & 3 & 17 & 3.5 & 1.7 & 8 \\ 0.05 & 10 & 17 & 0.1 & 8 & 14 \\ 3 & 3.5 & 1.7 & 10 & 17 & 8 \\ 17 & 8 & 0.05 & 10 & 0.1 & 14 \end{pmatrix}$$
$$P = (p_{ij}) = 10^{-4} \begin{pmatrix} 1312 & 1696 & 5569 & 124 & 8283 & 5886 \\ 2329 & 4135 & 8307 & 3736 & 1004 & 9991 \\ 2348 & 1451 & 3522 & 2883 & 3047 & 6650 \\ 4047 & 8828 & 8732 & 5743 & 1091 & 381 \end{pmatrix},$$

with $\mathbf{x}_0 = (0, 0, 0, 0, 0, 0)$.

4. Rosenbrock function

$$f(x_1, \dots, x_n) = \sum_{i=1}^{n-1} [100(x_{i+1} - x_i^2)^2 + (1 - x_i)^2] \quad n \geq 2.$$

with

- $\mathbf{x}_0 = (-1.2, 1) \in \mathbb{R}^2$.
- $\mathbf{x}_0 = (-1.2, 1, \dots, -1.2, 1) \in \mathbb{R}^{200}$.
- $\mathbf{x}_0 = (-1.2, 1, \dots, -1.2, 1) \in \mathbb{R}^{600}$.